

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Языки моделирования и описания цифровой аппаратуры»

Программная на SystemC и аппаратная на VerilogHDL реализации

“Преобразователь цифра - семисегментный дисплей”

Выполнил

студент гр. 5130904/40003

Николаев А.Д.

Руководитель

ст.преп

Петров А.В.

Санкт-Петербург

2026

Оглавление

Описание алгоритма	3
Программная реализация на SystemC алгоритма работы дисплея	3
Блок-схема работы	3
Текст программной реализации	4
main.cpp	4
SegDisplay.h	4
SegDisplay.cpp	5
SegDisplayTest.h	5
SegDisplayTest.cpp	6
SegDisplayRTL.h	7
SegDisplayRTL.cpp	7
Симуляция	7
Аппаратная реализация на Verilog HDL алгоритма работы дисплея	8
Блок схема	8
Код на языке Verilog транслированный с помощью sc2V	8
Симуляция	9
Проверка на синтезируемость	10
Отчёт в среде Quartus II	10
RTL схема	11
Технологическая схема	12

Описание алгоритма



Выводить E в случае ошибки. Двоичный код представлен в сокращенной записи (отбрасываются нули перед старшей единицей).

Входные сигналы:

sc_uint<4> digit

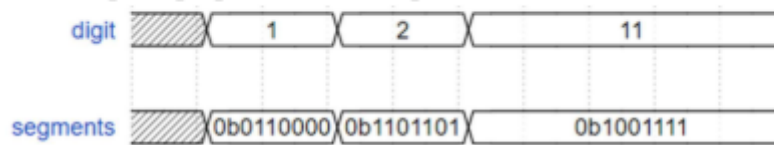
Выходные сигналы:

sc_uint<7> segments

Логика:

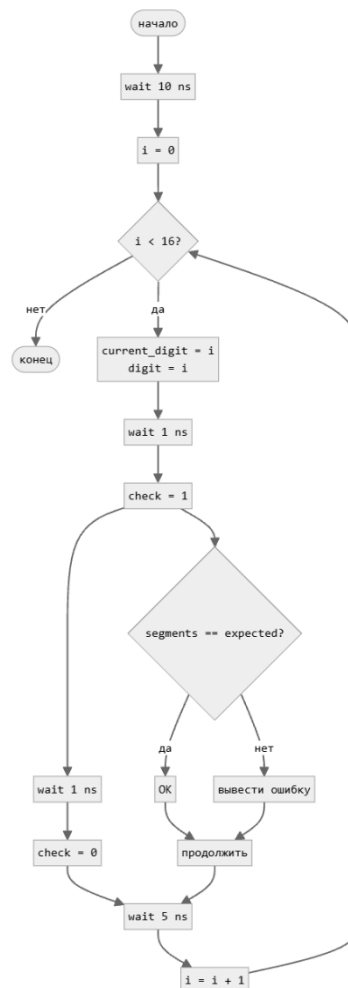
комбинационная

Пример временной диаграммы:



Программная реализация на SystemC алгоритма работы дисплея

Блок-схема работы



Текст программной реализации

main.cpp

```
1 | #include <systemc.h>
2 | #include "SegDisplayRTL.h"
3 | #include "SegDisplayTest.h"
4 |
5 | int sc_main(int argc, char* argv[])
6 | {
7 |     SegDisplayRTL segdisp("segdisp");
8 |     SegDisplayTest test("test");
9 |
10 |    sc_signal<sc_uint<4>> s_digit("digit");
11 |    sc_signal<sc_uint<7>> s_segments("segments");
12 |
13 |    test.digit(s_digit);
14 |    test.segments(s_segments);
15 |
16 |    segdisp.digit(s_digit);
17 |    segdisp.segments(s_segments);
18 |
19 |    sc_trace_file* tf = sc_create_vcd_trace_file("lab3");
20 |    tf->set_time_unit(1, SC_NS);
21 |
22 |    sc_trace(tf, s_digit, "digit");
23 |    sc_trace(tf, s_segments, "segments");
24 |
25 |    sc_start();
26 |
27 |    sc_close_vcd_trace_file(tf);
28 |
29 |    return 0;
30 | }
```

SegDisplay.h

```
1 | #ifndef SEGDISPLAY_H
2 | #define SEGDISPLAY_H
3 |
4 | #include <systemc.h>
5 |
6 | SC_MODULE(SegDisplay)
7 | {
8 |     sc_in<sc_uint<4>> digit;
9 |     sc_out<sc_uint<7>> segments;
10 |
11 |     void on_digit();
12 |
13 |     SC_CTOR(SegDisplay)
14 |     {
15 |         SC_METHOD(on_digit);
16 |         sensitive << digit;
17 |     }
18 | };
19 |
20 | #endif
```

SegDisplay.cpp

```
1 | #include "SegDisplay.h"
2 |
3 | void SegDisplay::on_digit()
4 | {
5 |     sc_uint<7> seg;
6 |
7 |     switch (digit.read())
8 |     {
9 |     case 0: seg = 0x7E; break; // 0b1111110
10 |    case 1: seg = 0x30; break; // 0b0110000
11 |    case 2: seg = 0x6D; break; // 0b1101101
12 |    case 3: seg = 0x79; break; // 0b1111001
13 |    case 4: seg = 0x33; break; // 0b0110011
14 |    case 5: seg = 0x5B; break; // 0b1011011
15 |    case 6: seg = 0x5F; break; // 0b1011111
16 |    case 7: seg = 0x70; break; // 0b1110000
17 |    case 8: seg = 0x7F; break; // 0b1111111
18 |    case 9: seg = 0x7B; break; // 0b1111011
19 |    default: seg = 0x4F; break; // 0b1001111 E
20 |    }
21 |
22 |    segments.write(seg);
23 | }
```

SegDisplayTest.h

```
1 | #ifndef SEGDISPLAYTEST_H
2 | #define SEGDISPLAYTEST_H
3 |
4 | #include <iostream>
5 | #include <systemc.h>
6 |
7 |
8 | SC_MODULE(SegDisplayTest)
9 | {
10 |     sc_in<bool>          clk;
11 |     sc_out<sc_uint<4> > digit;
12 |     sc_in<sc_uint<7> > segments;
13 |
14 |     sc_signal<bool>      check;
15 |
16 |     sc_uint<4>           current_digit;
17 |
18 |     void generate_digits();
19 |     void check_segments();
20 |
21 |     SC_CTOR(SegDisplayTest) :
22 |         current_digit(0)
23 |     {
24 |         SC_THREAD(generate_digits);
25 |         sensitive << clk.pos();
26 |
27 |         SC_METHOD(check_segments);
28 |         dont_initialize();
29 |         sensitive << check.posedge_event();
30 |     }
31 | };
```

```
32 |  
33 | #endif
```

SegDisplayTest.cpp

```
1 | #include "SegDisplayTest.h"  
2 |  
3 | void SegDisplayTest::generate_digits()  
4 | {  
5 |     check.write(0);  
6 |  
7 |     wait(10, SC_NS);  
8 |  
9 |     for (int i = 0; i < 16; i++)  
10 |    {  
11 |        current_digit = i;  
12 |        digit.write(i);  
13 |  
14 |        wait(1, SC_NS);  
15 |  
16 |        check.write(1);  
17 |        wait(1, SC_NS);  
18 |        check.write(0);  
19 |  
20 |        wait(5, SC_NS);  
21 |    }  
22 |  
23 |    sc_stop();  
24 | }  
25 |  
26 | void SegDisplayTest::check_segments()  
27 | {  
28 |     sc_uint<7> expected;  
29 |  
30 |     switch (current_digit)  
31 |     {  
32 |     case 0: expected = 0x7E; break;  
33 |     case 1: expected = 0x30; break;  
34 |     case 2: expected = 0x6D; break;  
35 |     case 3: expected = 0x79; break;  
36 |     case 4: expected = 0x33; break;  
37 |     case 5: expected = 0x5B; break;  
38 |     case 6: expected = 0x5F; break;  
39 |     case 7: expected = 0x70; break;  
40 |     case 8: expected = 0x7F; break;  
41 |     case 9: expected = 0x7B; break;  
42 |     default: expected = 0x4F; break;  
43 |     }  
44 |  
45 |     if (segments.read() != expected)  
46 |     {  
47 |         std::cerr << "Error at " << sc_time_stamp()  
48 |             << ": digit=" << current_digit  
49 |             << " expected=" << expected  
50 |             << " got=" << segments.read()  
51 |             << std::endl;  
52 |     }  
53 | }
```

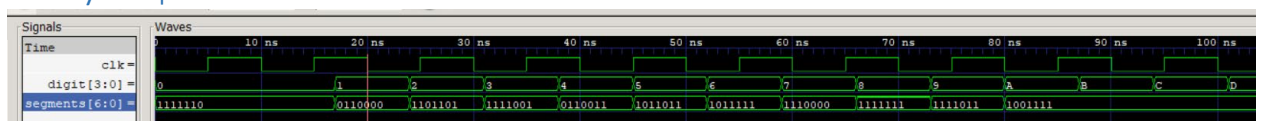
SegDisplayRTL.h

```
1 |  
2 | #include "systemc.h"  
3 |  
4 | SC_MODULE(SegDisplayRTL)  
5 | {  
6 |     sc_in< sc_uint<4> > digit;  
7 |     sc_out< sc_uint<7> > segments;  
8 |  
9 |     void comb_logic();  
10 |  
11 |     SC_CTOR(SegDisplayRTL)  
12 |     {  
13 |         SC_METHOD(comb_logic);  
14 |         sensitive << digit;  
15 |     }  
16 | };  
17 |
```

SegDisplayRTL.cpp

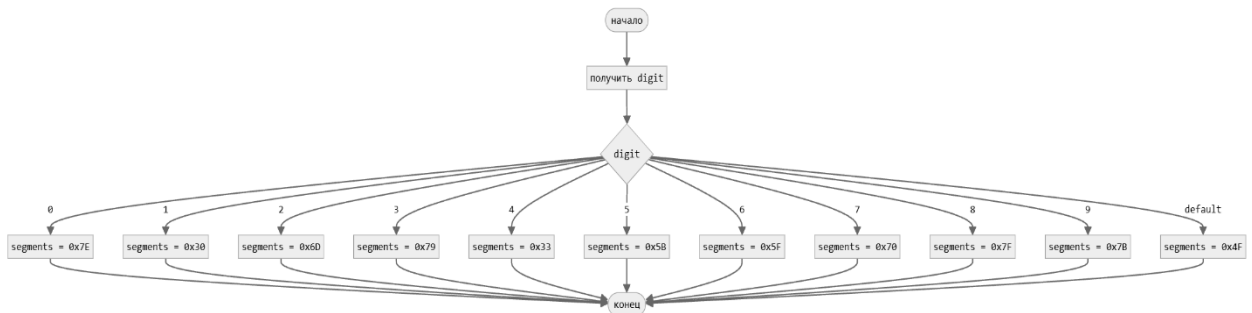
```
1 | #include "SegDisplayRTL.h"  
2 |  
3 | void SegDisplayRTL::comb_logic()  
4 | {  
5 |     sc_uint<7> seg;  
6 |  
7 |     switch (digit.read())  
8 |     {  
9 |         case 0: seg = 0x7E; break;  
10 |        case 1: seg = 0x30; break;  
11 |        case 2: seg = 0x6D; break;  
12 |        case 3: seg = 0x79; break;  
13 |        case 4: seg = 0x33; break;  
14 |        case 5: seg = 0x5B; break;  
15 |        case 6: seg = 0x5F; break;  
16 |        case 7: seg = 0x70; break;  
17 |        case 8: seg = 0x7F; break;  
18 |        case 9: seg = 0x7B; break;  
19 |        default: seg = 0x4F; break;  
20 |     }  
21 |  
22 |     segments.write(seg);  
23 | }
```

Симуляция



Аппаратная реализация на Verilog HDL алгоритма работы дисплея

Блок схема



Код на языке Verilog транслированный с помощью sc2V

```
1 | module SegDisplayRTL(segments,digit);
2 | output [6:0] segments;
3 | input [3:0] digit;
4 |
5 | reg [6:0] segments;
6 |
7 |
8 |
9 | //comb_logic:
10| reg [6:0] segcomb_logic;
11| always @( digit )
12|     begin
13|
14|         case(    digit )
15|
16|             0 :
17|             begin
18|                 segcomb_logic='h7E ;
19|             end
20|
21|             1 :
22|             begin
23|                 segcomb_logic='h30;
24|             end
25|
26|             2 :
27|             begin
28|                 segcomb_logic='h6D ;
29|             end
30|
31|             3 :
32|             begin
33|                 segcomb_logic='h79;
34|             end
35|
36|             4 :
37|             begin
38|                 segcomb_logic='h33;
39|             end
40|
41|             5 :
```

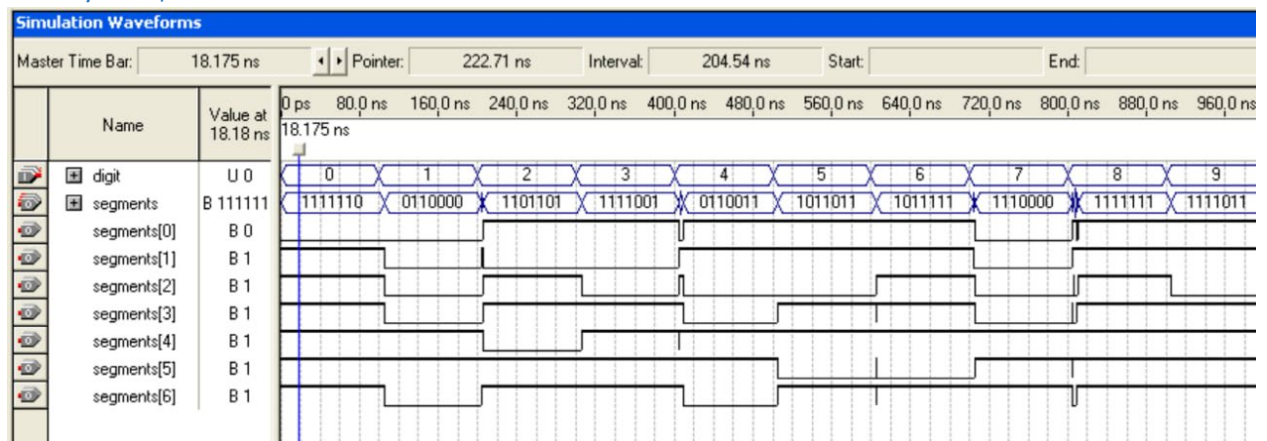


```

42|         begin
43|             segcomb_logic='h5B ;
44|         end
45|
46|         6 :
47|         begin
48|             segcomb_logic='h5F ;
49|         end
50|
51|         7 :
52|         begin
53|             segcomb_logic='h70;
54|         end
55|
56|         8 :
57|         begin
58|             segcomb_logic='h7F ;
59|         end
60|
61|         9 :
62|         begin
63|             segcomb_logic='h7B ;
64|         end
65|
66|         default:
67|         begin
68|             segcomb_logic='h4F ;
69|         end
70|
71|
72|     endcase
73|
74|
75|     segments = (segcomb_logic);
76|
77| end
78|
79| endmodule
80|

```

Симуляция

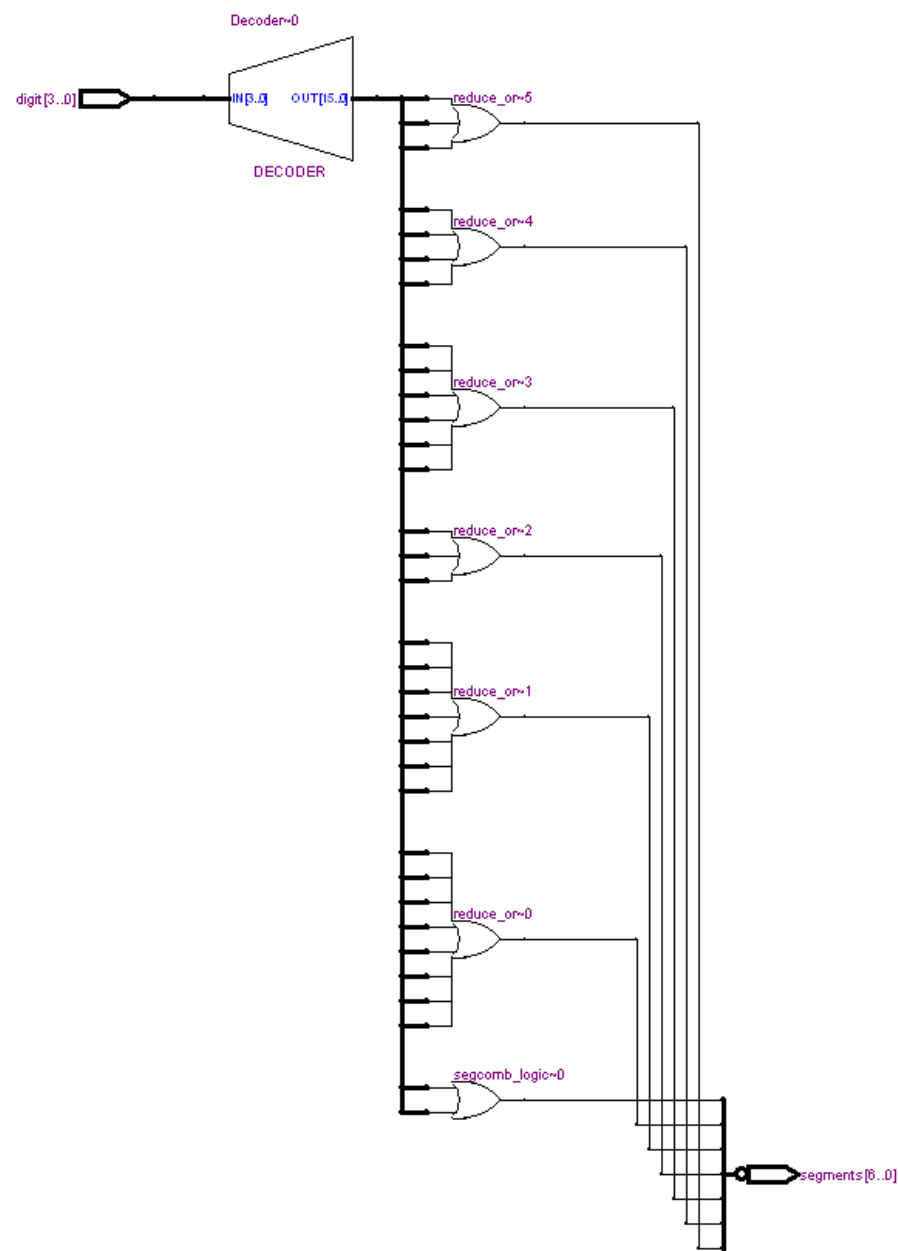


Проверка на синтезируемость

Отчёт в среде Quartus II

Flow Summary		
Flow Status	Successful - Wed May 04 16:51:15 2022	
Quartus II Version	5.0 Build 148 04/26/2005 SJ Full Version	
Revision Name	SegDisplayRTL	
Top-level Entity Name	SegDisplayRTL	
Family	Cyclone II	
Device	EP2C35F672C6	
Timing Models	Preliminary	
Met timing requirements	Yes	
Total logic elements	7 / 33,216 (< 1 %)	
Total registers	0	
Total pins	11 / 475 (2 %)	
Total virtual pins	0	
Total memory bits	0 / 483,840 (0 %)	
Embedded Multiplier 9-bit elements	0 / 70 (0 %)	
Total PLLs	0 / 4 (0 %)	

RTL cxema



Технологическая схема

